

OpenShift and Vault



Hybrid multi-cloud management is complex

Maintaining security across your cloud and on-premises environments can be challenging



71%

of organizations have or are planning multi-cloud deployments.¹



46%

of organizations lost revenue or customers due to a container or Kubernetes security incident.²



67%

of organizations delayed or slowed down deployment due to Kubernetes security concerns.²



Geopolitical data residency guidelines



Application availability



Disaster recovery



Edge deployments



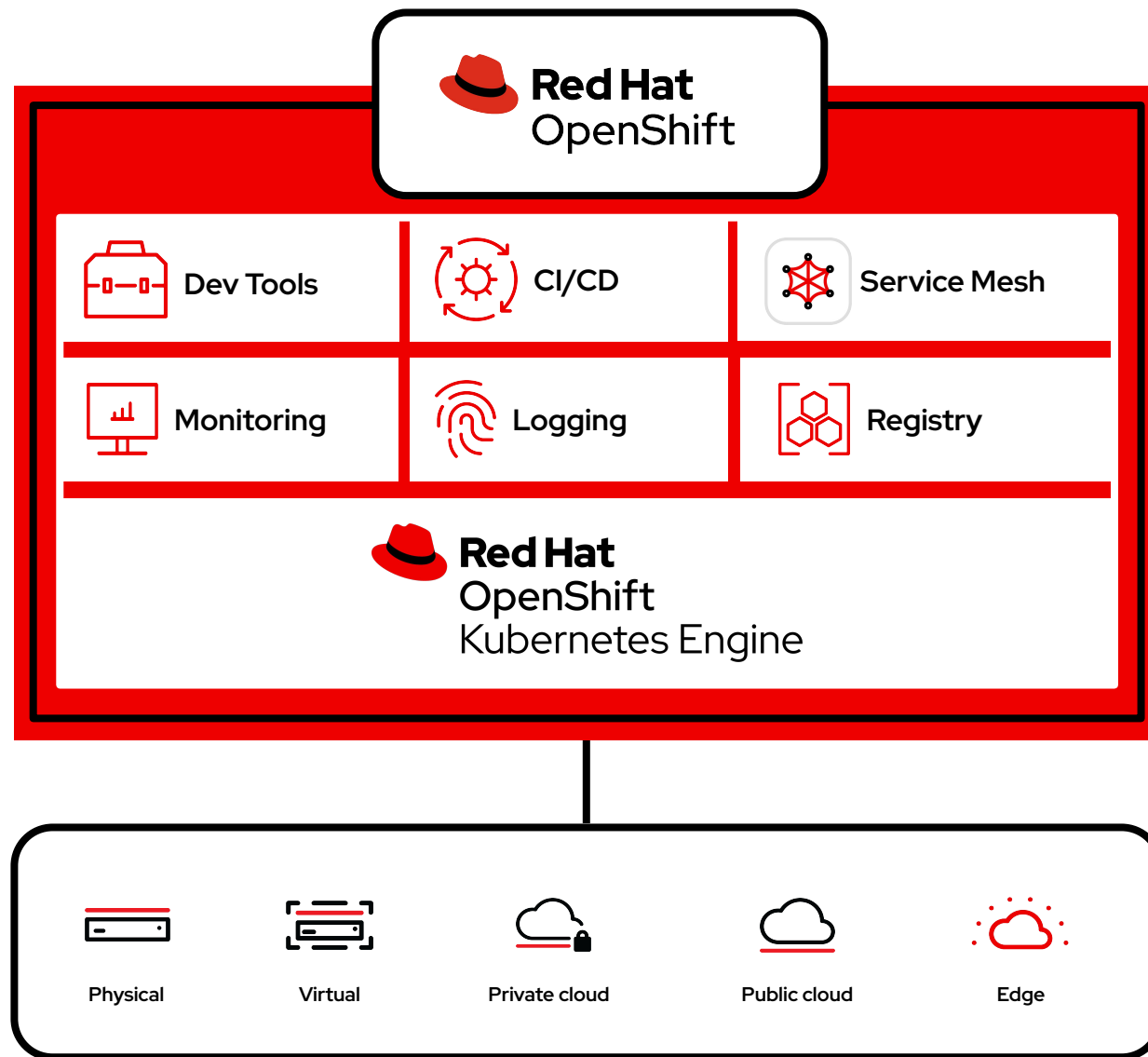
Address industry standards

Certificate management
Secrets management
Key management
Data protection
Authentication
Auth policies

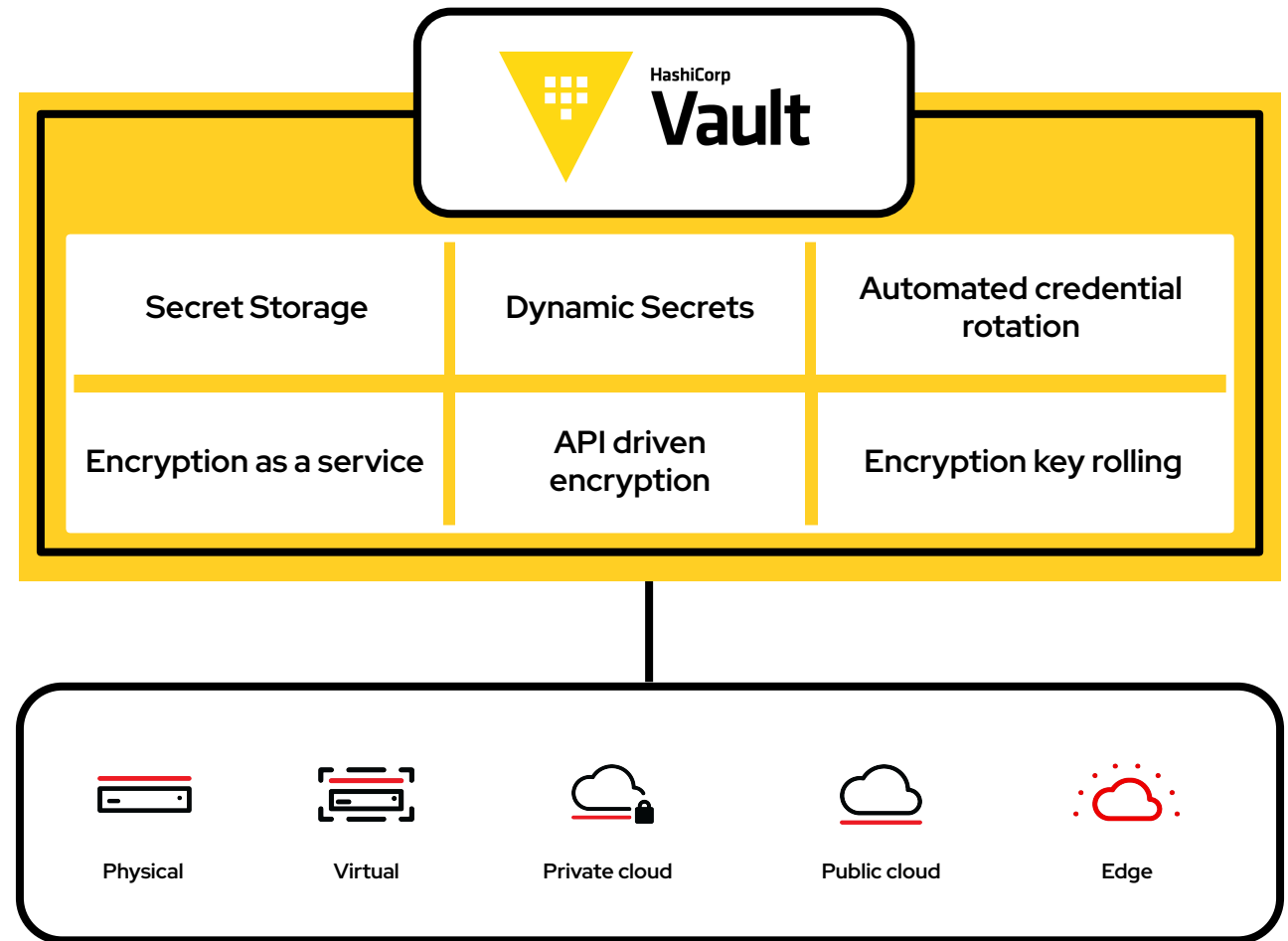
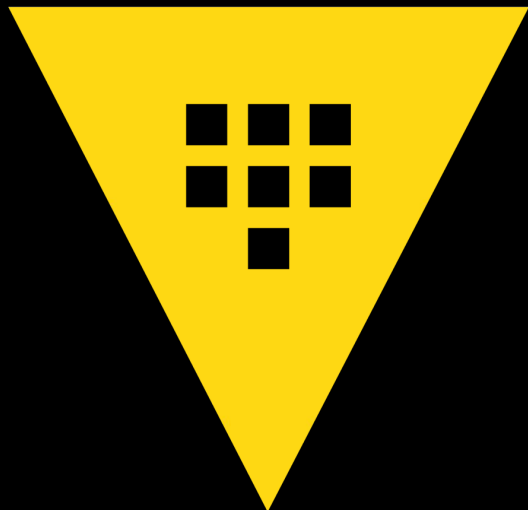
Organizations want to reduce complexity

Organizations want an integrated security solution to manage their most sensitive data and enforce policies consistently across different environments.

Red Hat OpenShift is a **Trusted Comprehensive Consistent** Application Platform



Manage secrets & protect sensitive data across Hybrid Cloud Environments with Vault





Deliver secure hybrid cloud applications

Protect secrets and data

End-to-end security at depth

Trusted container engine | Comprehensive application platform | Consistent across hybrid cloud | Protected workloads

Trusted enterprise Kubernetes engine

Built on Red Hat Enterprise Linux with verified software and Kubernetes for secure, mission-critical applications.

Comprehensive application platform

Build, deploy, secure, and monitor traditional, stateful, cloud-native, and AI-driven applications.

Consistent across hybrid cloud

Unified experience across data center, public cloud, multi-cloud, and edge, enhancing flexibility for new and modernized apps.

Identity-based security

IBM Vault is THE standard for security automation for access to secrets and sensitive data. It uses trusted identities to broker connections between applications and users.

Identity Access Management (IAM)

Use your providers to enforce that people and machines are who they say they are, and only get access to what they need to access.

Single centralized record

IBM Vault acts as a system of record for you to centrally store your secrets and sensitive data. Then, you can manage the lifecycle of those credentials over time.

Zero-trust architecture

Encrypt data in transit and at rest, with on-demand key and certificate management for microservices and service mesh.

Centralized access and certificate management

Manage secrets for containers and VMs with OpenShift across hybrid, multi-cloud, on-premises, and edge infrastructure.

Simplified secrets and certificate integration

Integrate with existing CI/CD and GitOps workflows to enhance application and platform security, improve developer experience, and streamline secure production paths.



Physical



Virtual



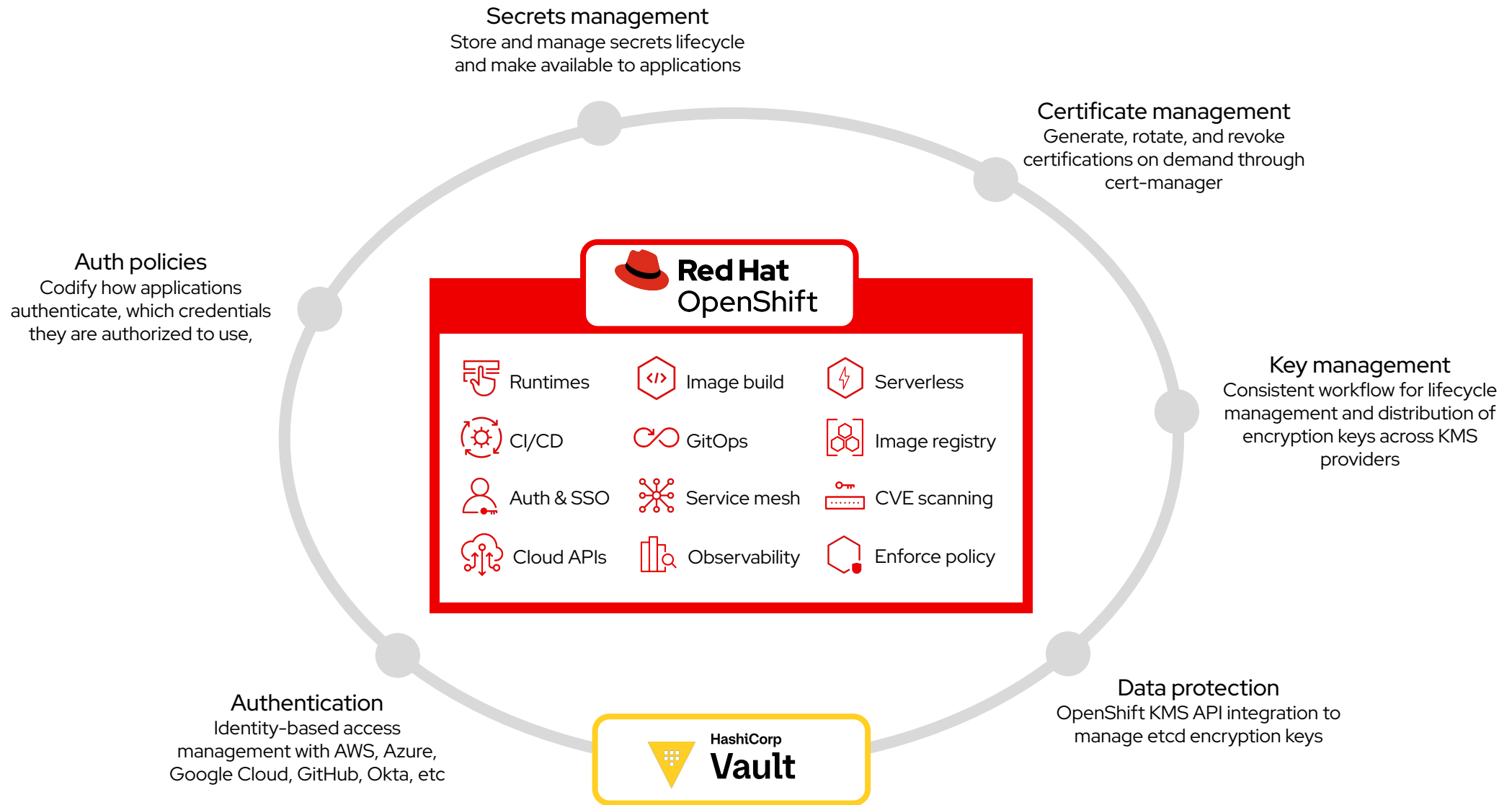
Private cloud



Public cloud



Edge



Technical Deep Dive

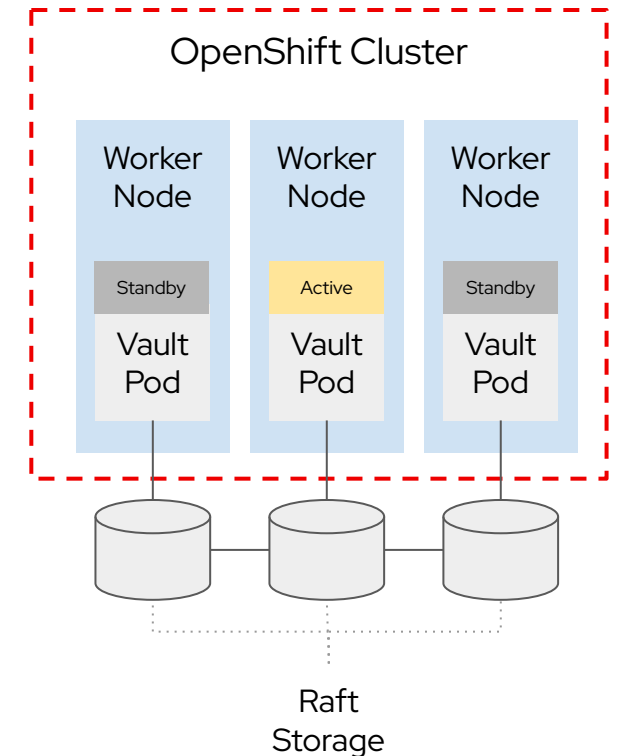
Protect Sensitive Data in OpenShift with Vault

- ▶ Deploy Vault in High Availability mode on OpenShift
- ▶ Manage OpenShift service accounts and roles with Vault
- ▶ Injects secrets into pods as CSI volumes with Secret Store CSI Driver
- ▶ Create and renew TLS certificates with cert-manager
- ▶ Protected sensitive data in GitOps workflows with Argo CD and Vault

Vault Deployment

Run Vault on OpenShift

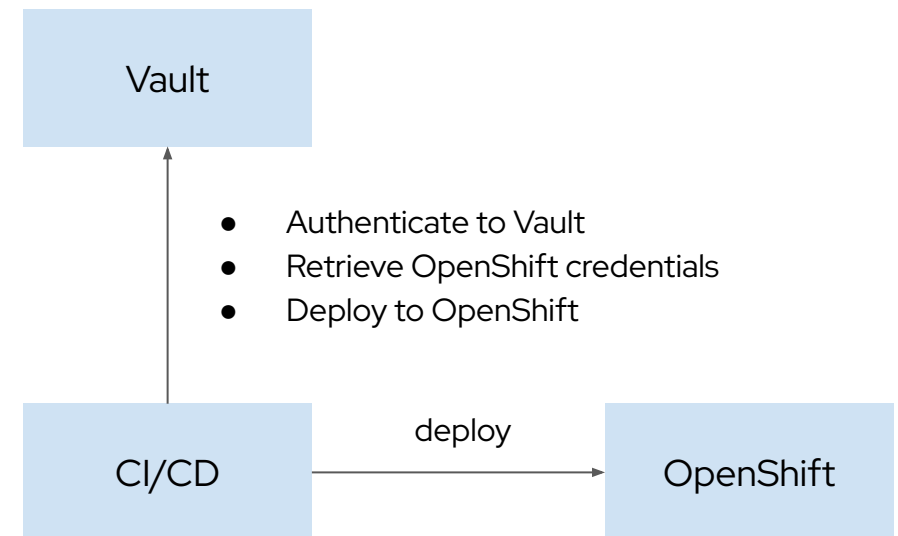
- Install through the Vault Helm Chart
 - Dev mode: single in-memory Vault server
 - Standalone:: a single Vault service with persistent storage
 - High Availability (HA): a cluster of Vault servers with persistent storage
- High Availability (HA) mode with auto-pilot Integrated Storage (Raft)
- Liveness and readiness probes for failure detection and rescheduling
- Leverage existing operational and security workflows for running Vault
- Workloads authenticate to Vault with OpenShift service account tokens



Managing OpenShift Service Accounts

Managing Service Accounts Roles

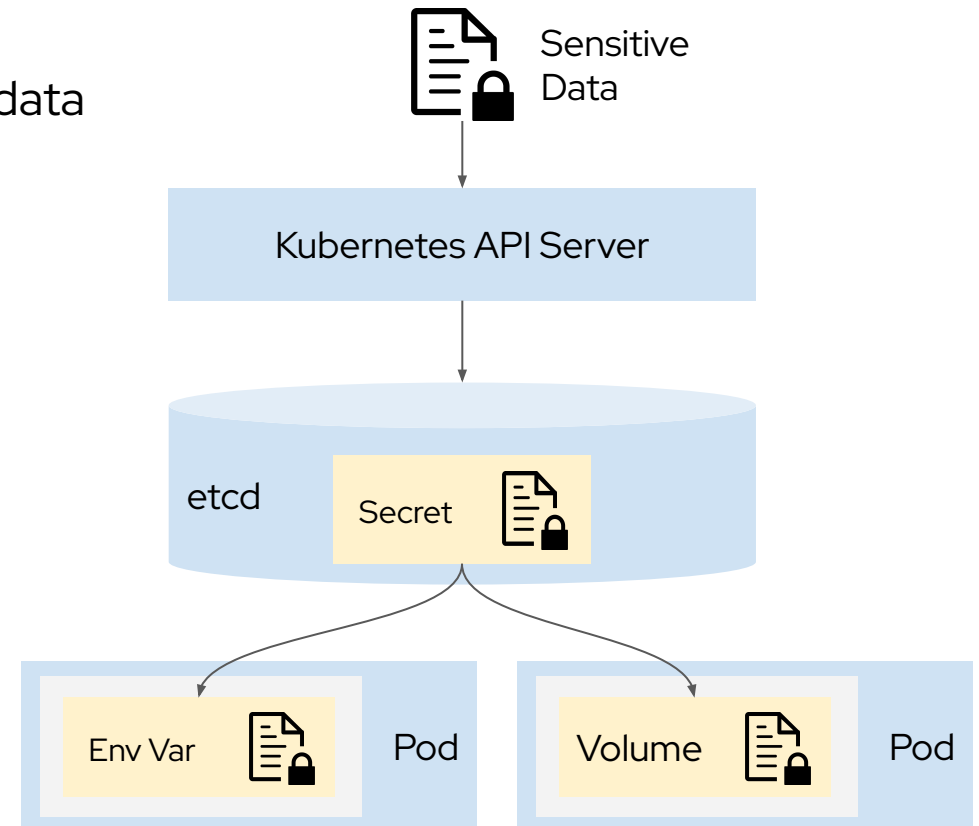
- Reduce exposure risk via short-lived service account tokens for accessing OpenShift
- Generate service account tokens with Vault
- Automate assignment of tightly scoped roles to service accounts
- Define time-to-live (TTL) to automatically expire and revoke access



Secret Management

Kubernetes Secrets

- Secret is a Kubernetes resource to store confidential data
 - Credentials
 - Certificates
 - API Tokens
 - SSH keys
- Separates confidential data from application code



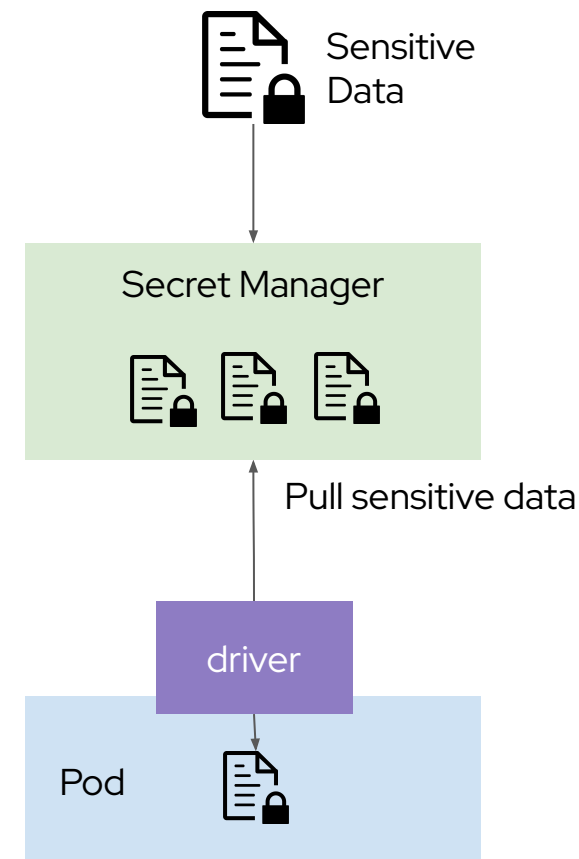
Security Challenges of Kubernetes Secrets

- **No encryption:** by default, secrets are stored in base64 encoded plain text
- **Access control:** misconfigured access control can allow unauthorized entities to access secrets within the namespace
- **Manual rotation:** manual and inconsistent key rotation can lead to stale or compromised credentials across clusters.
- **etcd Storage:** when not encrypted, secrets stored in etcd are vulnerable if the etcd database is compromised and when encrypted, the keys are accessible to cluster admins



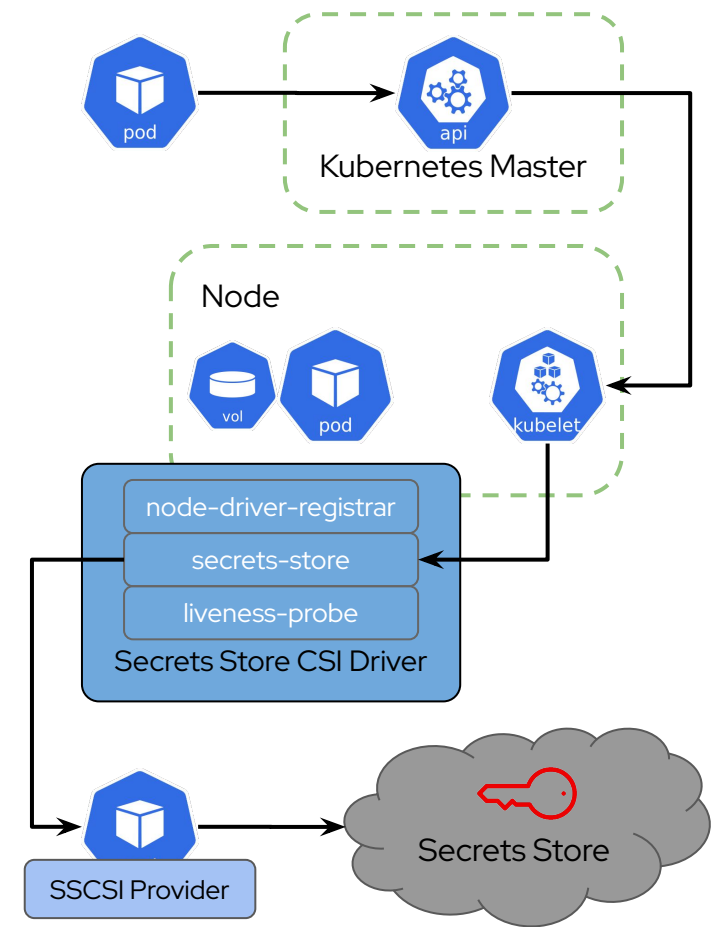
The Value of External Secret Management

- A dedicated system for managing sensitive data, designed to integrate with various platforms including Kubernetes.
- Store sensitive data outside Kubernetes
- Protect secrets from admin access
- Fine-grained access control
- Automated secret lifecycle management (e.g. rotation, expiration)



Secrets Store CSI Driver

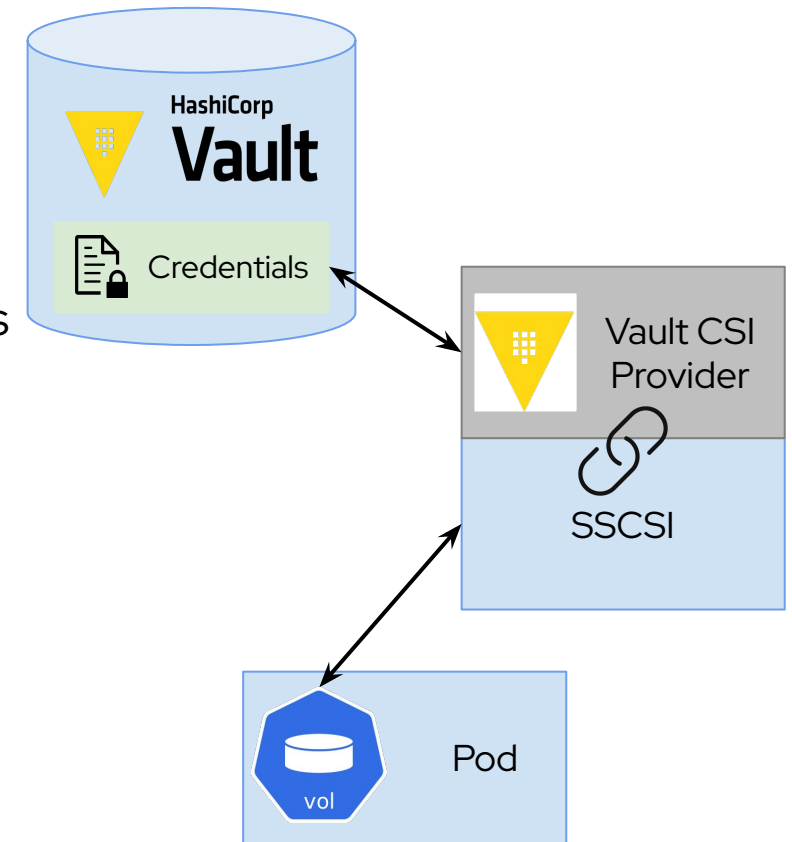
- **Day 2 Operator:** minimizes impact to cluster lifecycle
- **Workflow:** secrets from a secret store are mounted as a volume to a workload as an ephemeral volume.
 - Pod is created and scheduled on a node
 - Kubelet instructs CSI Driver to mount volume as tmpfs
 - SCSIS driver communicates with the SCSIS provider to fetch secrets via provider authentication method.
 - Retrieved secrets are sent to the SCSIS driver to be written to the volume filesystem.
 - Pod deletion will remove delete the mounted volume



* All network communication via GRPC

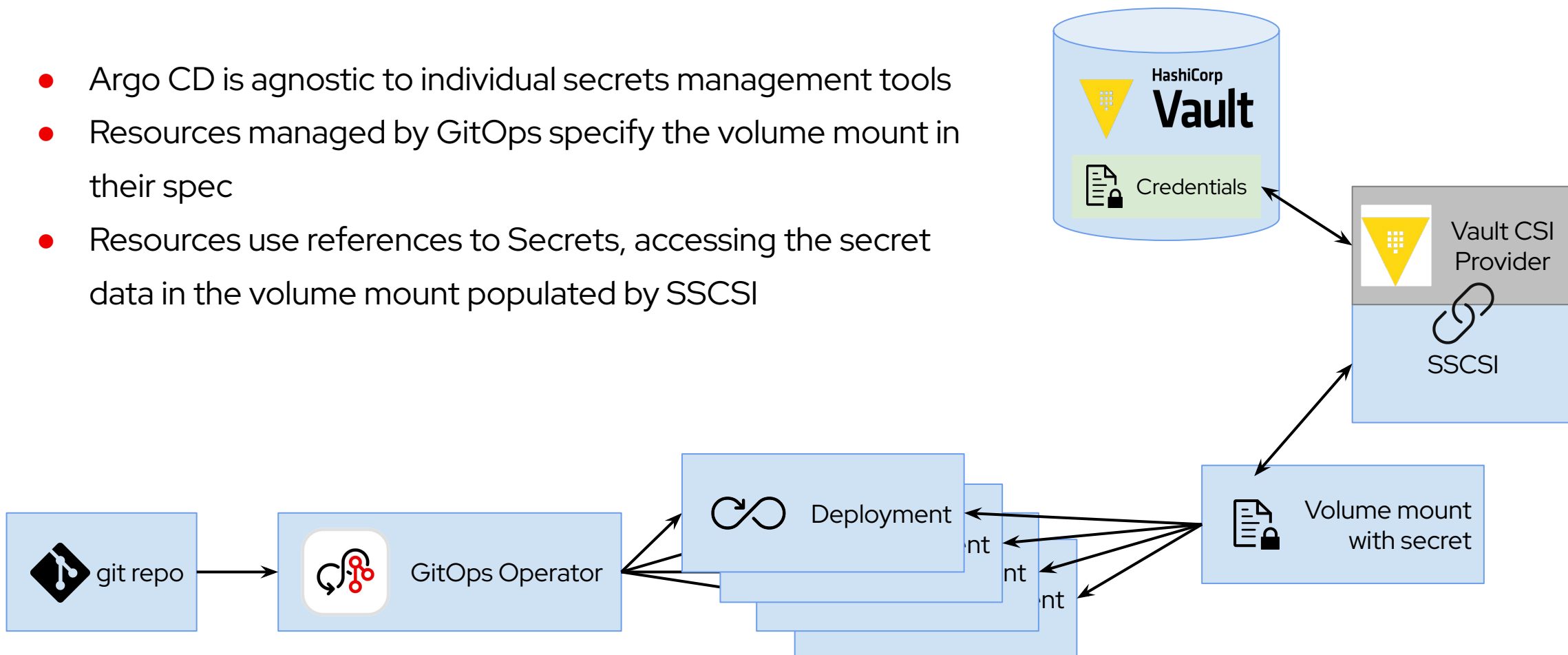
Key Features of SSCSI with Vault

- **HashiCorp Vault:** Provides increased security via short-lived secrets generators and enhanced cryptography for static secrets
- **Never store secrets on cluster:** enhance compliance with secrets mounted directly to workloads as ephemeral volumes
- **Multiple secrets, 1 volume:** Any secrets consumed by a workload can be referenced from a single location
- **Automatic rotation of secrets:** secrets updated in Vault are automatically propagated to workloads consuming the secret



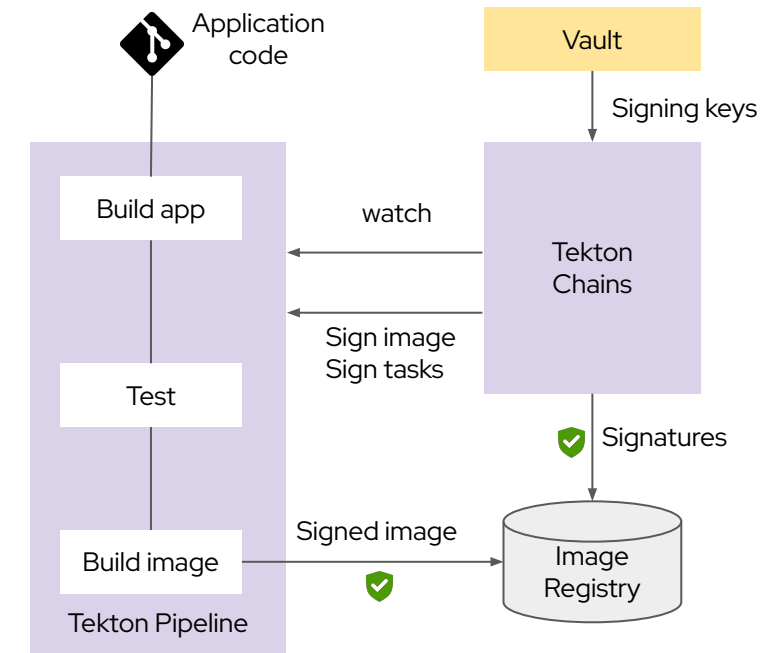
Secrets Management with OpenShift GitOps

- Argo CD is agnostic to individual secrets management tools
- Resources managed by GitOps specify the volume mount in their spec
- Resources use references to Secrets, accessing the secret data in the volume mount populated by SSCSI



Supply Chain Security with OpenShift Pipelines

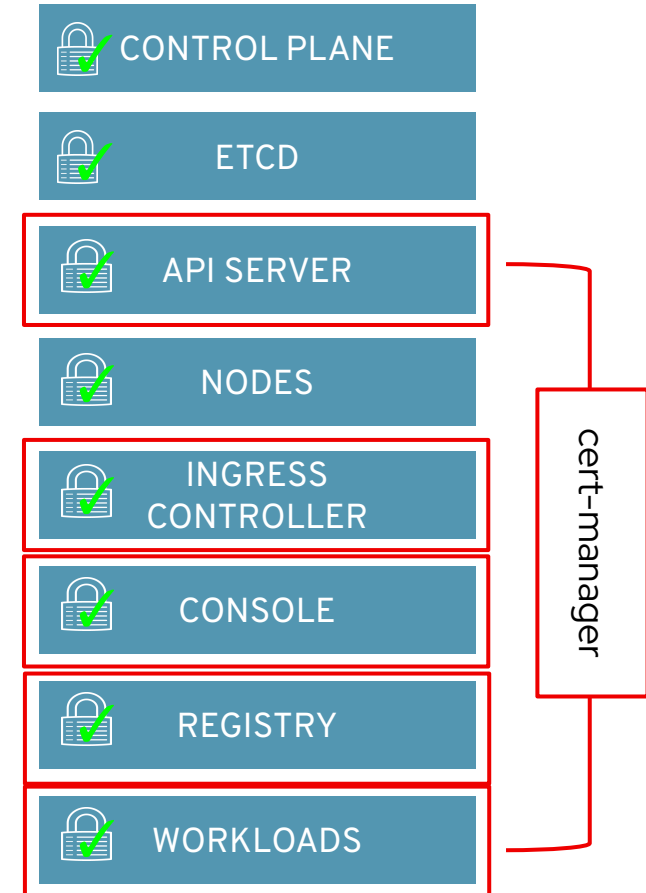
- Tekton Chains for supply chain security in Tekton Pipelines
- Automatically sign tasks, results and container images in pipelines
- Signatures stored in an OCI registry
- Publish provenance to transparency logs e.g. Rekor
- Supports x509, cosign, and external KMS providers
- When KMS provider such as IBM Vault used, signing keys are retrieved from the providers



Certificates Management

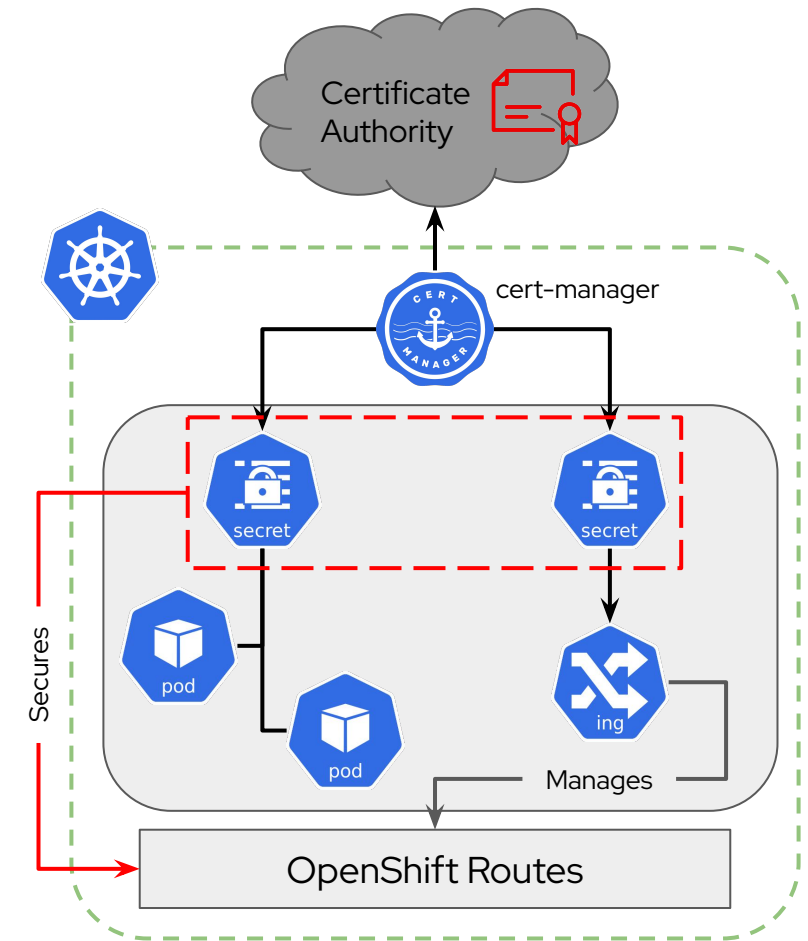
Protecting data in motion – Certificates in OpenShift

- Strong certificate use within OpenShift is critical to modern application security
- In OpenShift, certificates secure connections to:
 - master and nodes
 - Ingress controller and registry
 - Etcd
- Certificates are used to encrypt all traffic between control plane and worker nodes
- cert-manager adds **certificates and certificate issuers** as resource types in Kubernetes clusters, and **simplifies the process of obtaining, renewing and using** those certificates for:
 - Ingress, APIServer, application workloads



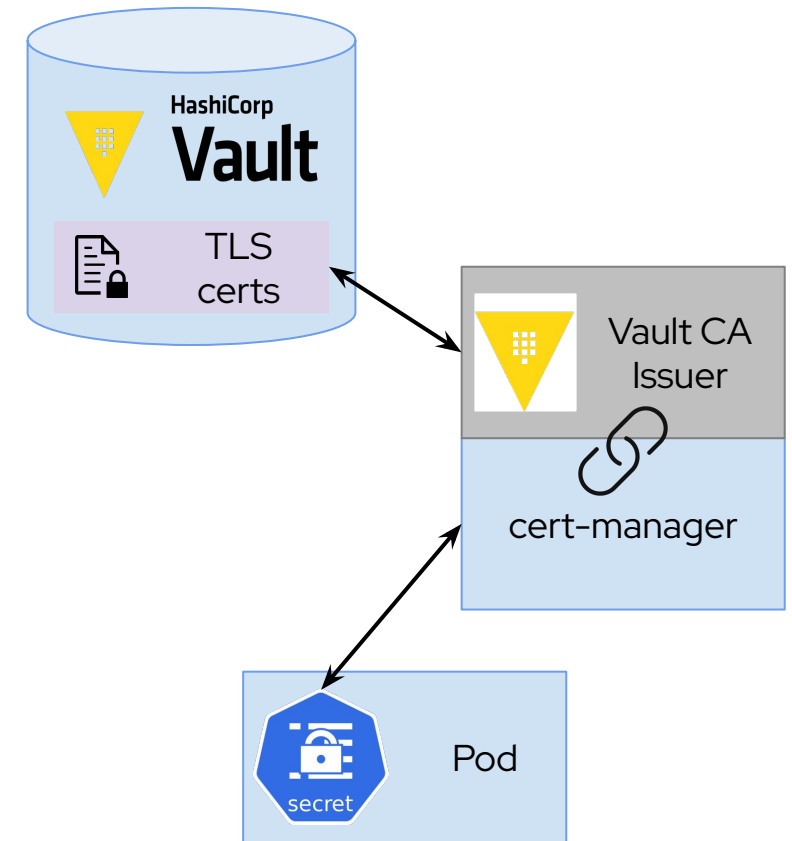
Cert-Manager

- **Day 2 Operator:** minimizes impact to cluster lifecycle
- **Workflow:** certificates from a external Certificate Authority (CA) are stored as Secrets on cluster, used to secure network connections
 - cert-manager operator watches custom Issuer and Certificate resources
 - New Certificate resources prompt request for a certificate
 - Issuer stores certificates obtained from the CA as Secrets
 - Secrets can be consumed by Applications or the Ingress controller to secure incoming network communications.



Key Features of Cert-Manager with Vault

- **HashiCorp Vault:** provides a single-source of truth for TLS certificates.
- **Certificate Management at Scale:** manage 100s to 1000s of certificates in production automatically.
 - Issuing and rotating certificates
- **Provide secure connections to:**
 - Master and nodes
 - Ingress controller and registry
- **Monitor usage:** enhanced compliance via monitoring of certificate issuance and expiry

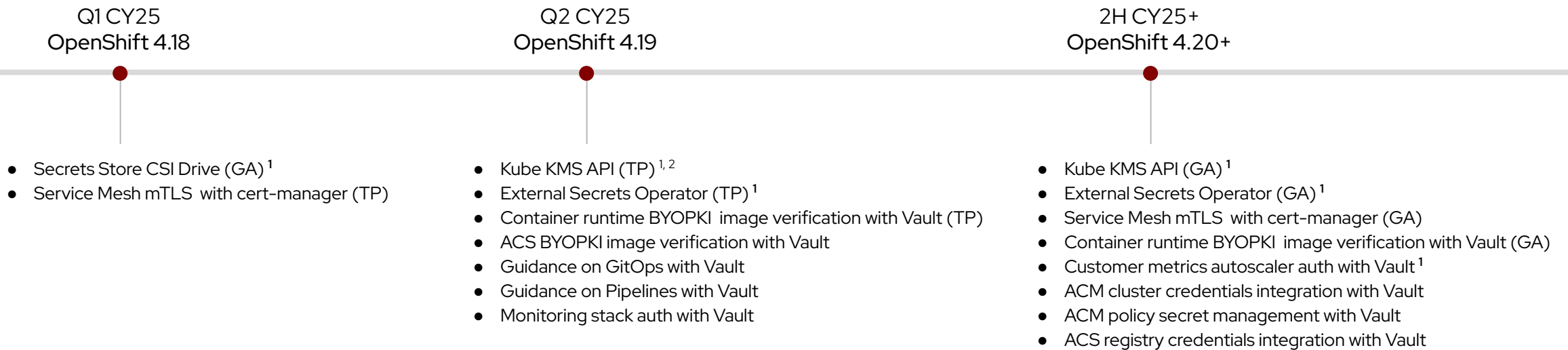


Roadmap

Secret Management Roadmap on OpenShift

Now	Next	Later
Enable data encryption at rest with external KMS Kube KMS API v2 support for storing etcs keys in an external KMS provider when encrypting data at rest Increase workloads security by consuming credentials and keys form Vault Secret Store CSI Driver and External Secrets to enable workloads to pull secrets from secret managers including Vault Manage certificates with Vault for securing Service Mesh workload communications Deliver istio-csr to enable using cert-manager with the Vault provider manage the lifecycle of TLS certificates used to secure communications in a service mesh	Ensure supply chain security through runtime image verification with Vault Enable verification of image signatures at runtime with PKI certificates pulled from Vault within container runtime and ACS CI/CD and GitOps interaction with Vault to increase security within application delivery Provide guides and docs for consuming keys, credentials and certificates from Vault within Tekton Pipelines and Argo CD and Argo Rollouts Strengthen operational security with observability stack pulling service credentials from Vault Enable the observability stack to use Vault for retrieving credentials used in third-party service communications Reduce risk of key exposure through storing encryption keys in Vault Vault KMS Plugin to integrate with Kube KMS API and facilitate storage and retrieval of etcd encryption keys in Vault	Simplify Vault operations and maintenance Deliver an operator to automate installation, upgrade Augment security posture with ACS storing sensitive assets in Vault Enable ACS to use Vault for retrieving cluster, image registries, and third-party service credentials Augment fleet management operational security with ACM integrations with Vault Enable ACM to use Vault for retrieving cluster, image registry, helm repository and third-party service credentials Improve OpenShift operational security with further integrations with Vault Customer metrics autoscaler and other OpenShift layered products to use Vault for accessing credentials and certificates

OpenShift with IBM Vault Integration Roadmap



¹ Commercial support for third-party providers and plugins e.g. AWS, Azure, Vault is the responsibility of the third-party

² Vault does not provide any plugins for Kube KMS and complete integration requires HashiCorp to develop an KMS plugin for Vault



Thank you

Red Hat is the world's leading provider of enterprise open source software solutions. Award-winning support, training, and consulting services make Red Hat a trusted adviser to the Fortune 500.

 linkedin.com/company/red-hat

 youtube.com/OpenShift

 facebook.com/redhatinc

 twitter.com/Openshift

What's next for OpenShift with IBM Vault?

Kube KMS API

Encrypt secrets and etcd data at rest using IBM Vault for storing and managing the encryption keys (*)

External Secrets Operator

Directly inject sensitive data from IBM Vault into OpenShift as native Kubernetes Secrets and enable applications to consume secrets through standard Kubernetes APIs

mTLS in Service Mesh

Enable using IBM Vault to provision signed TLS certificates for enabling mTLS for encrypting service communications with OpenShift Service Mesh

Container Runtime Image Verification

Enable container runtime (CRI-O) to use certificates managed by IBM Vault for verifying image signatures, allowing users to define policies that ensure only signed and trusted images can be pulled and deployed on OpenShift

Data encryption

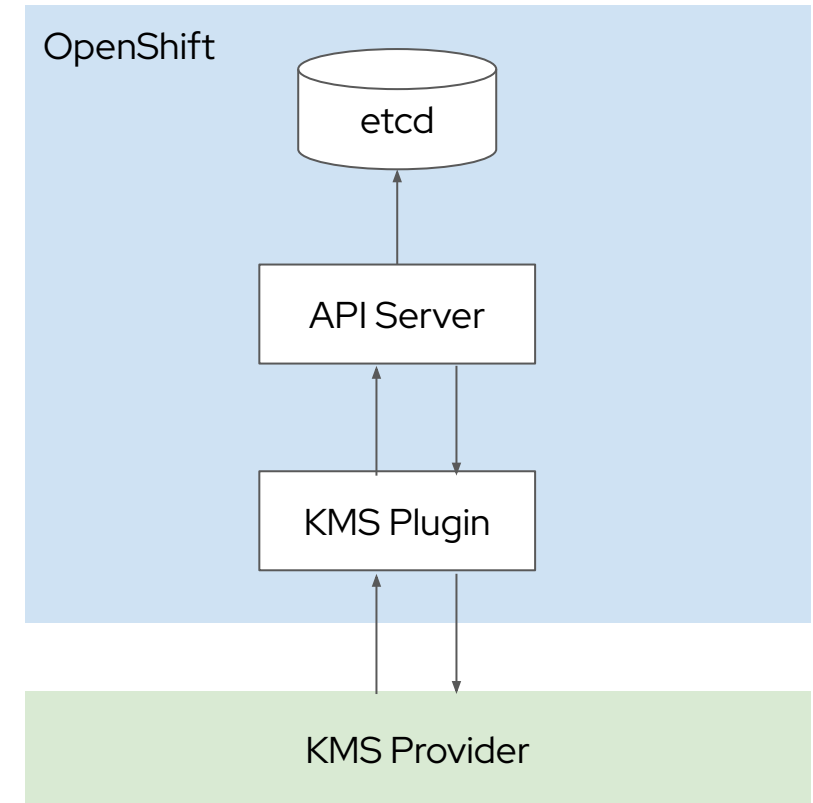


Encrypting etcd Data at Rest

- Data at rest refers to data in a persistent storage like etcd (e.g. Secrets, ConfigMaps, etc)
- Data encryption ensures the data is protected even if etcd gets compromised
- Data is encrypted before writing into etcd using an encryption key
- Data is decryption after reading from etcd
- Storing encryption keys
 - **Local storage:** encryption keys are stored on the host
 - **Managed storage:** encryption keys stored in an external key management system (KMS)

Encrypting etcd Data with KMS

- Kubernetes integrates with external KMS providers for etcd data encryption
- Envelope encryption scheme to encrypt data
- Data is encrypted using a data encryption key (DEK)
- DEKs are encrypted with a key encryption key (KEK) that is stored in KMS provider



Comparing Secrets Store CSI and External Secrets Operator

Secret Store CSI Driver solution:

Customer don't want secrets stored etcd as K8s secrets.

- Customers have **stringent compliance requirements** that make it necessary to store secrets and manage them ONLY in a central store vs on distributed clusters.
- Customers are bringing in workloads in an environment where **control plane is not managed by them**, so they **want full control of their workload secrets**. For e.g they bring in workloads in an MSP where they are a tenant, or they bring in their workload into *KS in Public clouds where the control plane is not managed by them.

External Secrets Operator: Customers who want secrets stored in etcd as K8s secrets

- Need **ease of integration** with platform and ease of use for developers
- Have **high degree of trust in control plane** of the cluster – the way etcd is managed (Encryption), RBAC
- Have **multi-cluster** hub< >spoke use cases for secrets management
- Need **cross-cluster** secrets integrations
- Need **Platform Secrets managed for non-application usage**, for e.g for Ingress, automation, image pull secrets
- The secrets need to be **modified on cluster with templating** for specific applications
- Need to **persist secrets on cluster**

How Does the Open Source Stack Up?

Feature	ESO	SSCSI	Sealed Secrets
Scalability	✓	✓ *	
GitOps Compatible	✓	✓	✓
Simplest Solution	✓ *		✓
Automatic Secret Rotation	✓ *	✓	
Secrets NOT stored on cluster		✓	
Advanced Secrets Templating	✓		
Control secrets in multiple namespaces	✓		
Use secrets for native K8S resources	✓		

ESO vs VSO

- **ESO + Vault Provider**
 - Operator watches for requests for CRDs as defined in by the External Secrets Operator.
 - Kubernetes native development allows for secrets to be consumed by any Kubernetes resource.
- **Vault Secret Operator**
 - Operator that watches for custom CRDs created specifically for Vault.
 - Tight integration with ALL Vault features, provides access to the entire Vault feature set.
- Both options create secrets in Kubernetes that can be used by workloads. ESO provides a more Kubernetes native approach. Additionally, ESO does not need cluster-wide permissions as compared to VSO.

SSCSI vs Vault Sidecar Agent Injector

- **SSCSI + Vault CSI Provider**
 - Operator watches for requests for csi driver `secrets-store.csi.k8s.io`.
 - Leverages native Kubernetes CSI driver under the hood `kubernetes-sigs`.
- **Agent-sidecar-injector**
 - Mutating webhook controller operates directly on etc.d, installed as a cluster service
 - Intercepts Create and Update requests to ApiServer and alters pod specifications based on the annotations present.
 - Provides greater integration with Vault authentication methods and secret templating
- Both options will mount secrets to a volume attached to a running pod. SSSCSI provides a more Kubernetes native approach. Additionally, SSSCSI does not require the same amount of privileges as the Agent Sidecar Injector since it does not need to operate directly on etc.d